Assignment 4: Advanced Image Generation

Liuyang Li | November 11, 2026

Part 1. CIFAR-10 generators and API

The project extends the existing course FastAPI with a diffusion model and an energy model. Both produce unconditional 32x32 RGB images. The earlier text, embedding, CNN and MNIST GAN routes are registered in the same root application.

Component	Implementation
Diffusion	Time-conditioned U-Net; three downsampling blocks; 200-step cosine DDPM schedule; Gaussian-noise MSE; EMA inference.
Energy model	Scalar energy CNN; persistent contrastive training; input-gradient sampling; frozen network parameters during sampling.
Deployment	Dockerfile and compose.yaml; app.main:app; packaged trained checkpoints; interactive /docs page.

```
docker compose up --build
# Open http://localhost:8000/docs
```

Method	Endpoint	Result
POST	/diffusion/generate	JSON with base64 RGB PNGs
POST	/energy/generate	JSON with base64 RGB PNGs
GET	/diffusion/generate.png	PNG grid
GET	/energy/generate.png	PNG grid
GET	/health	Readiness and training metadata

Gradient control

Diffusion training updates network parameters. During energy sampling, parameters are frozen and the current image requires gradients. Each update is detached before the next iteration. The later contrastive backward updates weights using detached negative images.

Verification and limits

Actual test results, training records and API evidence are under results/assignment4. The preparation environment has no Docker runtime and no authenticated GitHub account: container execution and the remote push remain separate steps. Small CPU runs demonstrate the workflow; generated images can remain noisy and do not establish convergence.

Part 2. Building Blocks of a Diffusion Model

Question 1. Sinusoidal time embedding

Let the dimension d be even, M = 10000, and j = 0, ..., d/2 - 1. Define the frequencies and sine/cosine pairs:

$$\omega_j = M^{-2j/d}, \qquad e_{2j}(t) = \sin(t\omega_j), \qquad e_{2j+1}(t) = \cos(t\omega_j).$$

Thus, the zero-indexed i-th coordinate is sin(t / M^(i/d)) when i is even, and cos(t / M^((i-1)/d)) when i is odd. Each pair shares a frequency. The angles are in radians. A learned MLP can subsequently transform this deterministic vector to condition the U-Net.

Convention: the calculations and model use sine-first interleaving with denominator d/2. Cosine-first grouped implementations reorder the same components. Implementations using d/2 - 1 instead change the actual frequencies, so an explicit convention is necessary.

Question 2. Embedding dimension 8 and timestep 1

Substitute d = 8 and t = 1:

$$\omega_j = 10000^{-2j/8} = 10000^{-j/4}, \qquad (\omega_0, \omega_1, \omega_2, \omega_3) = (1, 0.1, 0.01, 0.001).$$

Since t = 1, these are also the angles.

j	Angle	Sine	Cosine
0	1	0.84147098	0.54030231
1	0.1	0.09983342	0.99500417
2	0.01	0.00999983	0.99995000
3	0.001	0.00100000	0.99999950

The interleaved embedding is:

```
[0.84147098, 0.54030231, 0.09983342, 0.99500417,
 0.00999983, 0.99995000, 0.00100000, 0.99999950]
```

Reproducible calculation:

```
j = torch.arange(4, dtype=torch.float64)
frequencies = 10000 ** (-2 * j / 8)
angles = 1.0 * frequencies
embedding = torch.stack((angles.sin(), angles.cos()), dim=1).flatten()
```

Cosine-first grouped order instead gives [0.54030231, 0.99500417, 0.99995000, 0.99999950, 0.84147098, 0.09983342, 0.00999983, 0.00100000] for these same frequencies.

Question 3. Connection to transformer positional encoding

Both techniques represent a scalar index using sine and cosine functions at multiple frequencies. In the original transformer, the index is a token's position, and positional encodings are added to token embeddings so attention can use word order.

In diffusion, the index is the diffusion/noise timestep. Its embedding usually passes through an MLP and conditions several U-Net residual blocks through addition or feature modulation. The key difference is sequence position versus the global denoising timestep: a time embedding tells the denoiser which noise regime it must handle, rather than encoding each pixel's location.

Question 4. Bottleneck resolution

Assume each stride-2 block halves each spatial dimension, as for a 4x4 convolution with padding 1:

$$64 \times 64 \longrightarrow 32 \times 32 \longrightarrow 16 \times 16 \longrightarrow 8 \times 8.$$

After three downsampling blocks, 64 / 2^3 = 8. The bottleneck spatial resolution is therefore 8x8. Its channel count depends on the architecture. The question concerns a hypothetical 64x64 input; the implemented CIFAR-10 generator uses native 32x32 images and a 4x4 bottleneck.

Question 5. U-Net output and loss

In a noise-prediction DDPM, the U-Net receives the noisy image x_t and timestep t and outputs predicted Gaussian noise epsilon_theta(x_t, t). Its output has shape B x C x H x W, matching the input image. It is not a class label or scalar energy.

Define alpha_t = 1 - beta_t and alpha_bar_t as the product of alpha_s for s = 1 through t. Sample clean image x_0, timestep t, and epsilon from N(0, I), then construct:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon.$$

The sampled noise is the known target. Compare it with the prediction:

$$L_{\text{simple}} = \mathbb{E}_{x_0, t, \varepsilon}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|_2^2\right].$$

The implementation averages squared errors over the batch and pixels. Backpropagation updates network parameters; the image, timestep and sampled noise are not optimizer parameters.

```
t = torch.randint(T, (images.shape[0],), device=images.device)
noise = torch.randn_like(images)
a = alpha_bar[t][:, None, None, None]
x_t = a.sqrt() * images + (1 - a).sqrt() * noise
predicted_noise = unet(x_t, t)
loss = torch.nn.functional.mse_loss(predicted_noise, noise)
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
```

Clean-image and velocity prediction are alternatives, but this implementation specifically predicts noise. Its array indices for diffusion steps start at zero.

Part 3. Building Blocks of an Energy Model

Question 6. Basic gradient calculations

(a) Differentiate y = x^2 + 3x:

$$\frac{dy}{dx} = 2x + 3, \qquad \frac{dy}{dx}\Big|_{x=2} = 2(2) + 3 = 7.$$

The code prints x.grad = tensor([7.]).

(b) With requires_grad=False on x, this particular expression has no input that requires gradients. Thus y.requires_grad is False and y.grad_fn is None. Calling y.backward() raises:

```
RuntimeError: element 0 of tensors does not require grad and does not have a grad_fn
```

x.grad remains None. A different expression can still be tracked if another input, such as a trainable weight, requires gradients.

(c) torch.tensor defaults to requires_grad=False. Omitting the flag does not enable gradient tracking for the new tensor:

```
x = torch.tensor([2.0])
print(x.requires_grad)  # False
```

Question 7. Introduce weights

Write y = w_0 x^2 + w_1 x with x = 2, w_0 = 1 and w_1 = 3.

(a) In the supplied code, x requires gradients but w does not. The result is:

```
x.grad = tensor([7.])
w.grad = None
```

PyTorch can use the numerical values of w to calculate dy/dx without accumulating derivatives into w. Calling a tensor a weight does not make it trainable. Standard neural network layer weights are normally nn.Parameter objects, which require gradients by default.

(b) The partial derivatives are:

$$\frac{\partial y}{\partial w_0} = x^2 = 4, \qquad \frac{\partial y}{\partial w_1} = x = 2.$$

Enable gradients on w before the forward computation:

```
x = torch.tensor([2.0], requires_grad=True)
w = torch.tensor([1.0, 3.0], requires_grad=True)
y = w[0] * x**2 + w[1] * x
y.backward()
print("x.grad =", x.grad)
print("w.grad =", w.grad)
```

Output:

```
x.grad = tensor([7.])
w.grad = tensor([4., 2.])
```

If only weight gradients are needed, x can have requires_grad=False; w still receives [4, 2].

(c) As in Question 6(c), a new torch.tensor defaults to requires_grad=False.

Question 8. Breaking the graph

z = y.detach() disconnects z from the graph leading to y and x. Both z and w consequently do not require gradients, so w.backward() raises the error in Question 6(b). The numerical forward value remains 6; its gradient connection has been removed.

Retain z while removing detach:

```
x = torch.tensor([1.0], requires_grad=True)
y = x * 3
z = y                  # z = y.clone() also preserves the connection
w = z * 2
w.backward()
print("x.grad =", x.grad)
```

By the chain rule:

$$\frac{dw}{dx} = \frac{dw}{dz}\frac{dz}{dy}\frac{dy}{dx} = 2 \cdot 1 \cdot 3 = 6.$$

The corrected code prints x.grad = tensor([6.]). Setting z.requires_grad_(True) after detaching creates a new leaf but does not reconnect it to x.

Detaching between iterations is intentional in EBM sampling: each updated image becomes a new optimization variable and the full sampling trajectory need not be differentiated.

Question 9. Gradient accumulation

PyTorch adds gradients into leaf tensors' .grad fields. The first backward contributes d(2x)/dx = 2. The second contributes d(3x)/dx = 3:

$$g_1 = 2, \qquad g_2 = g_1 + 3 = 2 + 3 = 5.$$

The original code prints:

```
After first backward: x.grad = tensor([2.])
After second backward: x.grad = tensor([5.])
```

Clear the gradient to obtain independent derivatives:

```
x = torch.tensor([1.0], requires_grad=True)
y1 = x * 2
y1.backward()
print(x.grad)           # tensor([2.])
x.grad = None          # x.grad.zero_() is another option here

y2 = x * 3
y2.backward()
print(x.grad)           # tensor([3.])
```

In neural network training, optimizer.zero_grad(set_to_none=True) clears gradients of that optimizer's parameters. It does not clear an arbitrary input tensor unless the input belongs to the optimizer. Accumulation can be intentional when combining microbatches. Here the two expressions have separate forward graphs, so retain_graph=True is unnecessary and would not clear accumulated gradients.

References

1. PyTorch, torch.tensor: https://docs.pytorch.org/docs/stable/generated/torch.tensor.html

2. PyTorch, Tensor.detach: https://docs.pytorch.org/docs/stable/generated/torch.Tensor.detach.html

3. PyTorch, Autograd mechanics: https://docs.pytorch.org/docs/stable/notes/autograd.html

4. Vaswani et al. (2017), Attention Is All You Need, Section 3.5: https://arxiv.org/abs/1706.03762

5. Ho, Jain and Abbeel (2020), Denoising Diffusion Probabilistic Models: https://arxiv.org/abs/2006.11239

6. OpenAI, guided-diffusion time embedding:
https://github.com/openai/guided-diffusion/blob/main/guided_diffusion/nn.py

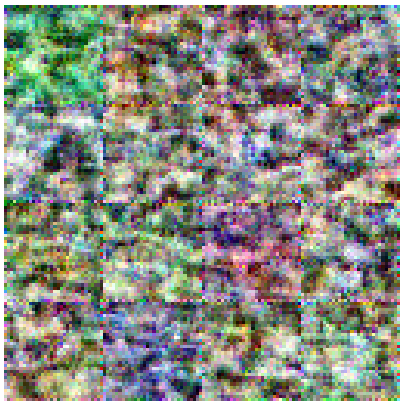
7. UvA, Deep Energy-Based Generative Models:
https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/tutorial8/Deep_Energy_Models.html

Training records and generated samples

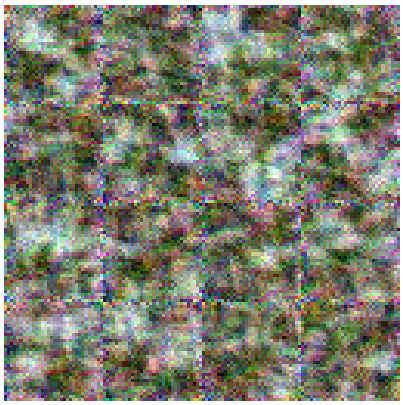
These compact CPU models use eight base channels, native CIFAR-10 training images and the stated objectives. Labels are ignored. These early training results are included to show the functioning pipeline. Visual quality remains limited; no FID, Inception Score or convergence claim is made.

Model	Epochs	Optimizer steps	Images seen
diffusion	3	1173	150,000
energy	3	1173	150,000

Diffusion API samples, seed 123



Energy API samples, seed 123



Training histories and checkpoint metadata are authoritative. The Colab notebook supports longer training with a larger 32-channel model.